AI Supply Chain Breach

Understanding the 2026 Litellm AI Supply Chain Breach

18 Aug

¿Qué ocurrió?

Se trata de un **ataque a la cadena de suministro de software** que tuvo lugar el **24 de marzo de 2026**. La cadena de eventos fue así:

1. **Un grupo de hackers llamado "TeamPCP"** comprometió el pipeline de CI/CD de **Trivy**, una herramienta de escaneo de vulnerabilidades de código abierto muy popular.
2. A través de ese acceso, **robaron los tokens de publicación de LiteLLM en PyPI** (el repositorio de paquetes de Python).
3. Con esos tokens, publicaron **dos versiones maliciosas de LiteLLM: la 1.82.7 y la 1.82.8**, que estuvieron disponibles en PyPI durante aproximadamente **40 minutos** (desde las 10:39 UTC del 24 de marzo de 2026) antes de ser retiradas.
4. Esas versiones maliciosas contenían un **roboautomático de credenciales** que, al ejecutarse, recolectaba:
   Claves de AWS (`AWS_SECRET_ACCESS_KEY` , credenciales en `~/.aws/credentials`)
   Tokens de Kubernetes, claves SSH, claves de bases de datos, certificados SSL privados
   Variables de entorno completas (que en CI/CD suelen contener secretos)
   Historial de shell y configuraciones de CI/CD
5. Los datos robados se enviaban cifrados a un servidor controlado por los atacantes (`models.litellm.cloud`).

El artículo de Hudson Rock afirma que obtuvieron un **archivo de 153 GB con 433.909 archivos**, incluyendo **118.829 volcados de runners de CI/CD** atribuibles a **~2.488 dominios corporativos**, exponiendo AWS keys, tokens de API y credenciales de bases de datos de muchas organizaciones.

¿estoy afectado?

¿Debería preocuparse tu organización que usa AWS?

Depende de si usaron LiteLLM durante la ventana de exposición. Esto es lo clave:

🔴 Sí, deben preocuparse SI:

- Alguien hizo `pip install litellm` **sin fijar versión** el 24 de marzo de 2026 entre las 10:39 y las 11:19 UTC (aproximadamente).
- Tienen pipelines de CI/CD, Dockerfiles o imágenes que instalan LiteLLM sin versión fija y se construyeron en esa ventana.
- Un proyecto o dependencia transitoria (frameworks de agentes de IA, MCP servers, herramientas de orquestación de LLM) pudo haber arrastrado LiteLLM como dependencia sin versión fija.

En ese caso, **las credenciales de AWS que existieran en ese entorno deben considerarse comprometidas**. El malware recolectaba específicamente claves de AWS.

🟢 Probablemente no hay riesgo SI:

- No usan LiteLLM en absoluto.
- Usan LiteLLM pero con **versiones fijadas anteriores** (1.82.6 o menor) y no instalaron nada en esa ventana de tiempo.
- Sus pipelines no ejecutan `pip install litellm`.

¿que deberia hacer?

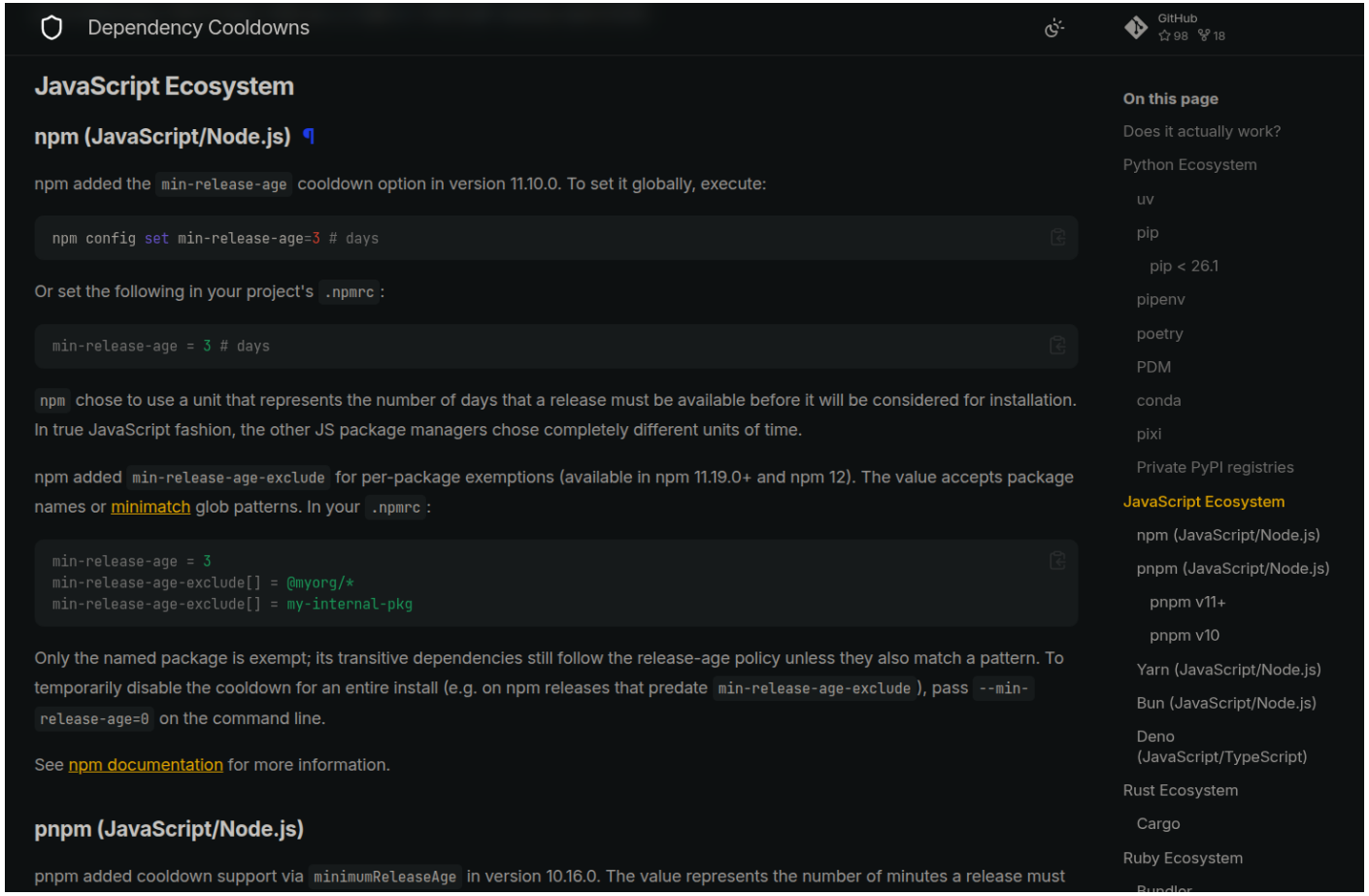
¿Qué deberían hacer? (Acciones recomendadas)

Independientemente del nivel de riesgo, estas son las medidas concretas:

1. **Verificar si instalaron las versiones afectadas**: revisen logs de CI/CD y cachés de pip para confirmar que no se instaló `litellm==1.82.7` o `1.82.8`.
2. **Si hay confirmación o duda razonable — rotar credenciales de AWS YA**:
   Invalidar y recrear todas las **IAM Access Keys** que hayan estado en entornos de CI/CD.
   Revisar **AWS CloudTrail** buscando actividad anómala desde el 24 de marzo de 2026.
   Revisar hallazgos de **AWS GuardDuty** y **AWS Config**.
3. **Inspeccionar persistencia**: buscar archivos `.pth` sospechosos en `site-packages` y servicios systemd no autorizados (el malware instalaba un backdoor persistente).
4. **Rotar otros secretos** que pudieran estar en esos entornos: tokens de GitHub/GitLab, passwords de bases de datos, certificados SSL.
5. **A futuro**: fijar versiones de dependencias, usar credenciales de corta duración (IAM roles temporales en lugar de access keys fijas), y aplicar filtrado de salida (egress filtering) en runners de CI/CD.

Lecciones aprendidas

- https://cooldowns.dev/



IC

Criptografía

Índice de temas

- Encoding
- Criptografía Clasica
- Hashing
- Criptografía Moderna (simétrica)

Codificación

- El encoding de caracteres es usado para representar un repertorio de caracteres en algún tipo de sistema de codificación.
- En ocasiones nos referimos al encoding como la conversión de la representación de los caracteres de un sistema de codificación a otro.

> **Algunos ejemplos de sistemas de codificación**

```
- ASCII
- Representación numérica en distintas otras bases
- Unicode
- EBCDIC
- Morse
- Braile
- Lenguaje de señas
- Base64
```

ASCII

American **S**tandard **C**ode for **I**nformation **I**nterchange es un metodo de codificación de 7-bit character usado en Windows, UNIX, and Macintosh machines.

representación numerica

```
- base  2 o binario
    01000011 01101001 01100010 01100101 01110010 01110011 01100101 01100111

- base 10 o decimal           67  105   98  101  114  115  101  103
- base 16 o hexadecimal     0x43 0x69 0x62 0x65 0x72 0x73 0x65 0x67
```

Binario	Dec	Hex	Representación	Binario	Dec	Hex	Representación	Binario	Dec	Hex	Representación
0010 0000	32	20	espacio ( )	0100 0000	64	40	@	0110 0000	96	60	`
0010 0001	33	21	!	0100 0001	65	41	A	0110 0001	97	61	a
0010 0010	34	22	"	0100 0010	66	42	B	0110 0010	98	62	b
0010 0011	35	23	#	0100 0011	67	43	C	0110 0011	99	63	c
0010 0100	36	24	$	0100 0100	68	44	D	0110 0100	100	64	d
0010 0101	37	25	%	0100 0101	69	45	E	0110 0101	101	65	e
0010 0110	38	26	&	0100 0110	70	46	F	0110 0110	102	66	f
0010 0111	39	27	'	0100 0111	71	47	G	0110 0111	103	67	g
0010 1000	40	28	(	0100 1000	72	48	H	0110 1000	104	68	h
0010 1001	41	29	)	0100 1001	73	49	I	0110 1001	105	69	i
0010 1010	42	2A	*	0100 1010	74	4A	J	0110 1010	106	6A	j
0010 1011	43	2B	+	0100 1011	75	4B	K	0110 1011	107	6B	k
0010 1100	44	2C	,	0100 1100	76	4C	L	0110 1100	108	6C	l
0010 1101	45	2D	-	0100 1101	77	4D	M	0110 1101	109	6D	m
0010 1110	46	2E	.	0100 1110	78	4E	N	0110 1110	110	6E	n
0010 1111	47	2F	/	0100 1111	79	4F	O	0110 1111	111	6F	o
0011 0000	48	30	0	0101 0000	80	50	P	0111 0000	112	70	p
0011 0001	49	31	1	0101 0001	81	51	Q	0111 0001	113	71	q
0011 0010	50	32	2	0101 0010	82	52	R	0111 0010	114	72	r
0011 0011	51	33	3	0101 0011	83	53	S	0111 0011	115	73	s
0011 0100	52	34	4	0101 0100	84	54	T	0111 0100	116	74	t
0011 0101	53	35	5	0101 0101	85	55	U	0111 0101	117	75	u

Criptografía

¿Qué es?

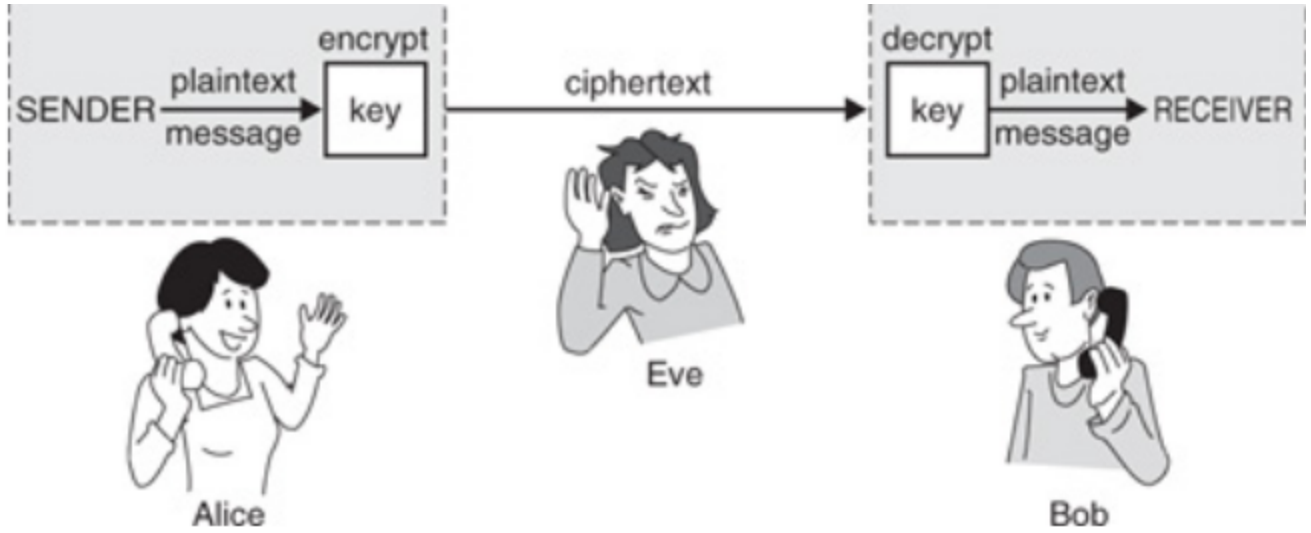
La criptografía (del griego “ocultar” y “escribir”), literalmente “escritura oculta”, es el arte o ciencia de cifrar y descifrar información utilizando técnicas que hagan posible el intercambio de mensajes de manera segura de forma tal que sólo puedan ser leídos por las personas a quienes van dirigidos.

Cifrar

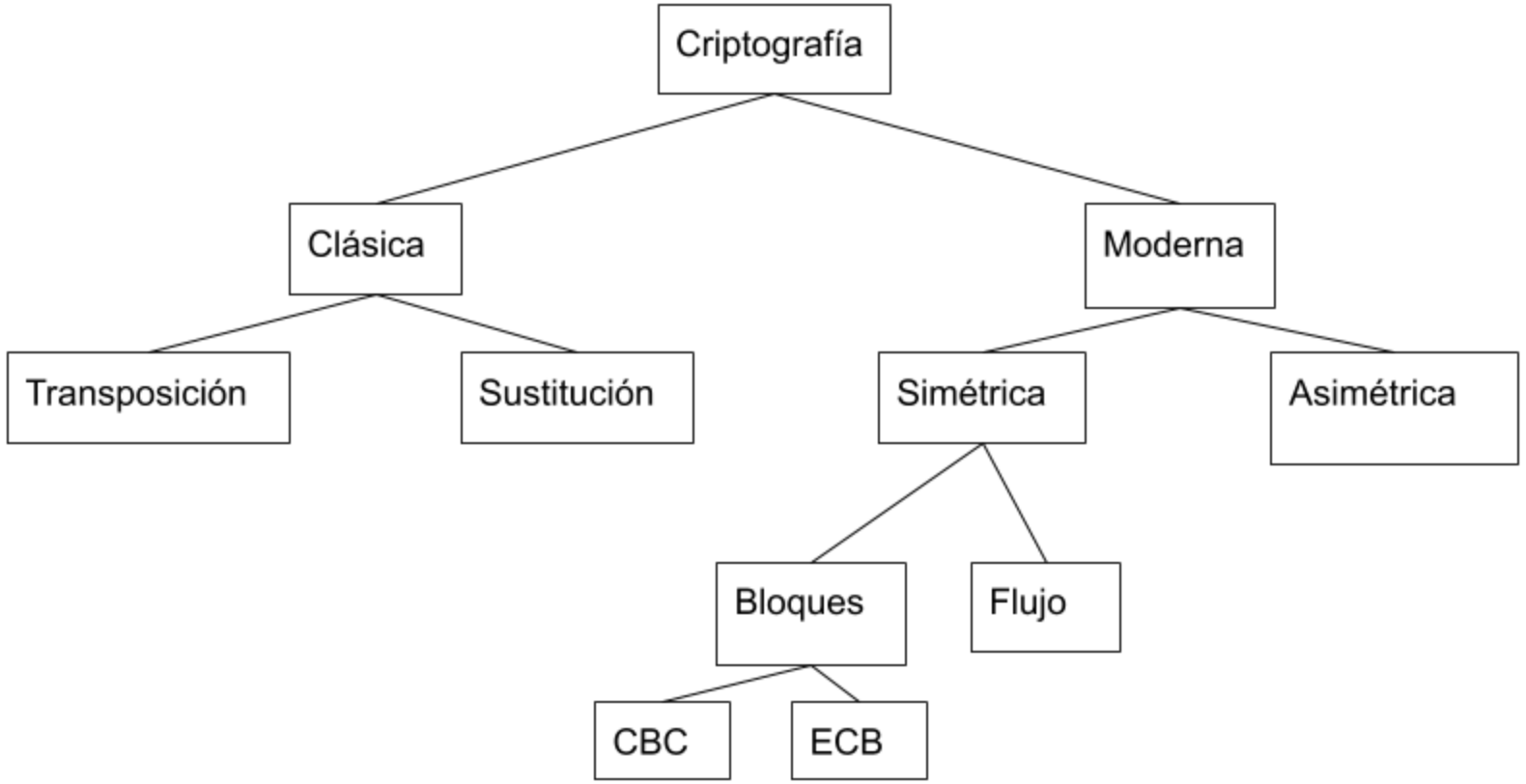
Conversión de un mensaje legible (**texto plano** o **plaintext**) a un dato sin sentido aparente (**mensaje cifrado** o **ciphertext**).

Esta conversión requiere del uso de una **clave** o **key**.

El emisor (creador del mensaje cifrado) y los destinatarios deben compartir tanto la técnica de descifrado como la clave a utilizar.



Criptografía clásica y moderna



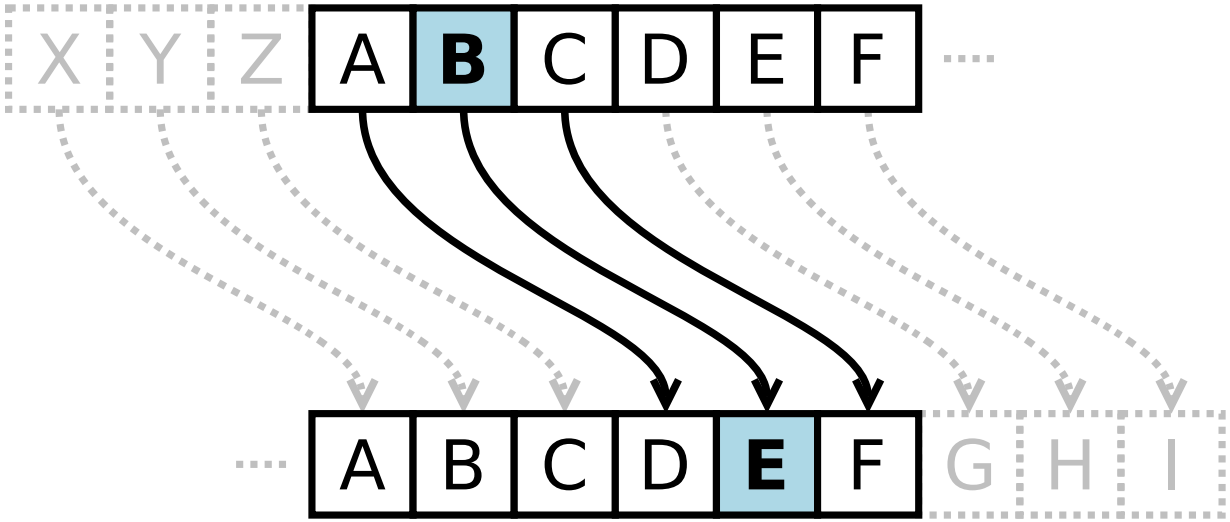
Criptografía Clásica

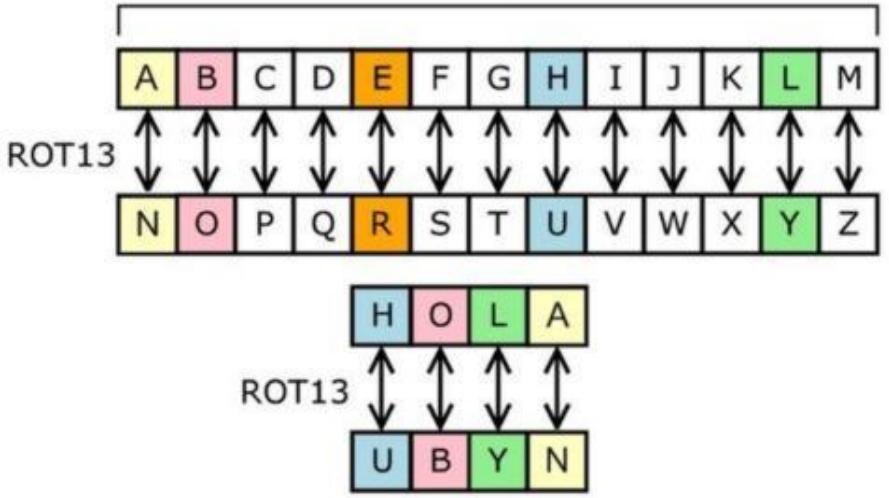
- Muy antigua
- Sin computadoras
- Se ocultaba el algoritmo

> Algunos ejemplos
>
> - ROT
> - Scytale / Escítala
> - Columnar Transposition
> - Máquina Enigma (2da guerra mundial)

ROT 3 - Cifrado Caesar

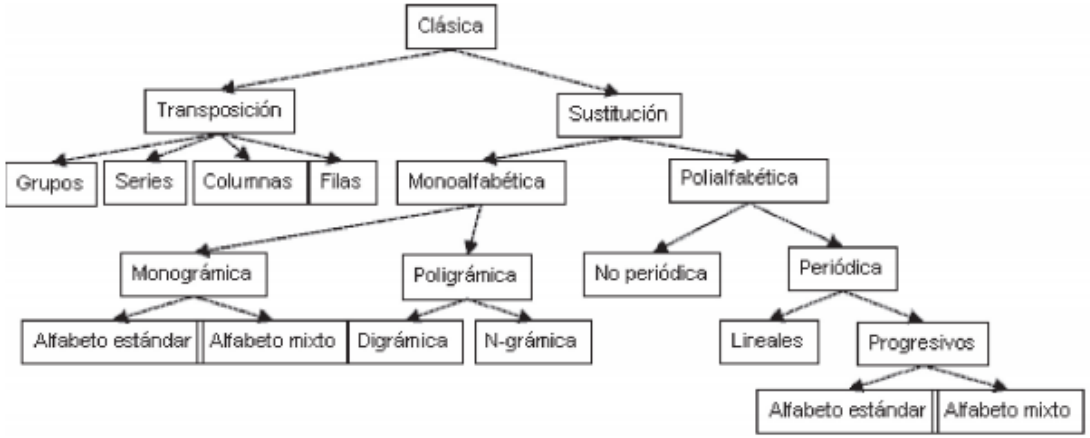
El cifrado César recibe su nombre en honor a Julio César, que, según Suetonio, lo usó con un desplazamiento de tres espacios para proteger sus mensajes importantes de contenido militar





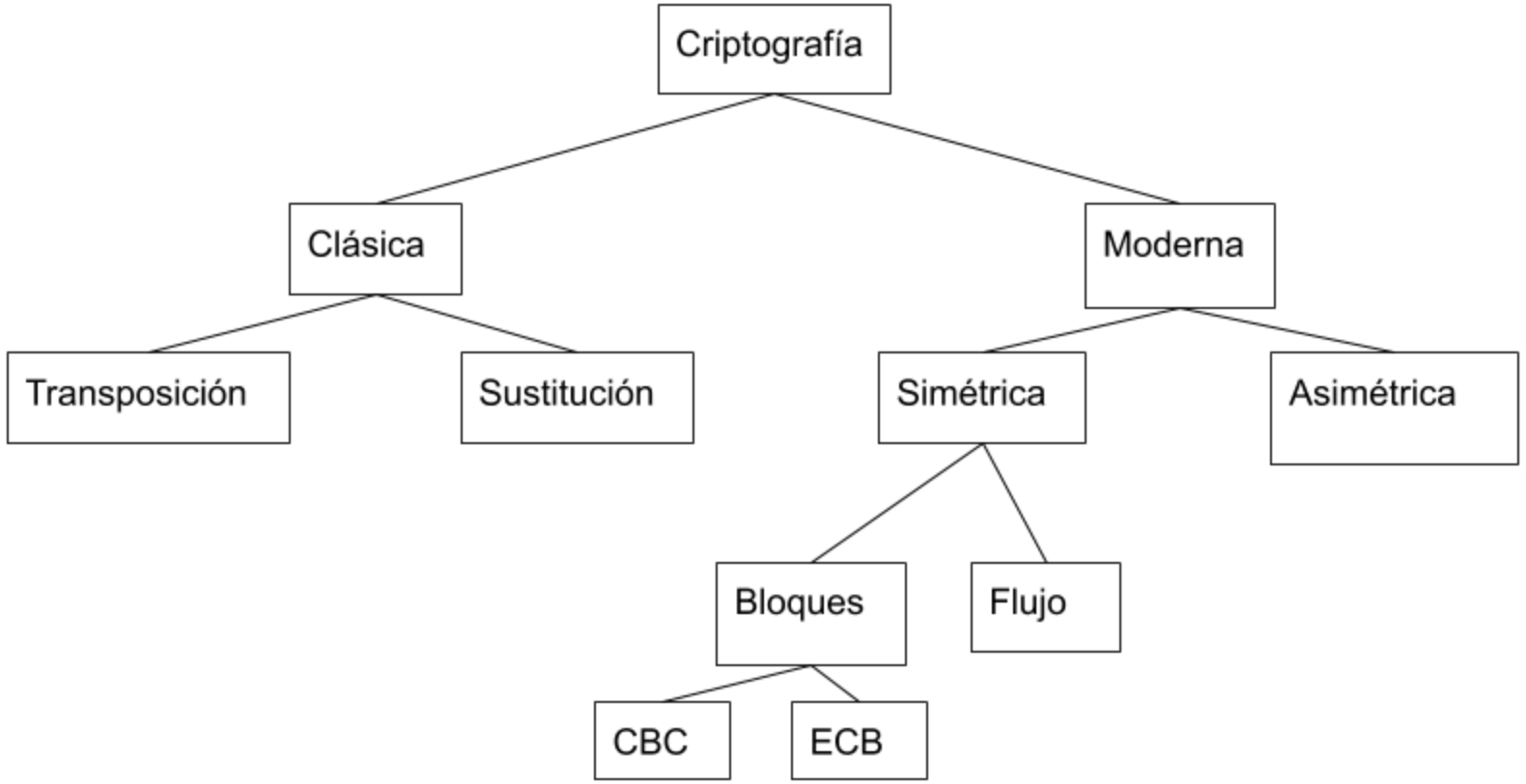
Criptografía clásica

Antes de la computadora era importante ocultar el algorítmo para no comprometer el mensaje.



 ... todo cambió cuando aparecieron las computadoras

Criptografía Moderna



Sistemas de criptografía

Existen dos tipos básicos de criptosistemas modernos:

> - **Sistemas de cifrado simétrico** (también conocidos como sistemas de clave secreta o clave privada)
> - **Sistemas de cifrado asimétrico** (también conocidos como sistemas de clave pública)

> Un concepto muy importante utilizado en criptografía moderna es el de las funciones de HASH.

(Funciones de hash)

- Son funciones que transforman una entrada y retornan una string de salida de longitud fija, conocido como “message digest”.
- Propiedades de las funciones de hash:
  - Determinístico.
    El mismo mensaje siempre da el mismo hash
  - No son reversibles.
    No es posible obtener el mensaje original a partir del hash
  - Un pequeño cambio en un mensaje cambia completamente el valor del hash o message digest
  - Es rápido calcular el hash para un mensaje dado

Algunas funciones de hash

```
MD5   (128 bists, RFC 1321)
SHA-1 (160 bits, NIST FIPS 180-2)
SHA-2 (SHA-224, SHA-256, SHA-384, SHA-512)
SHA-3 (SHA-224, SHA-256, SHA-384, SHA-512)
```

MD5

```
$ echo "Ciberseguridad" | md5sum
0e6e40411263311de0fa956a73cb2130  -

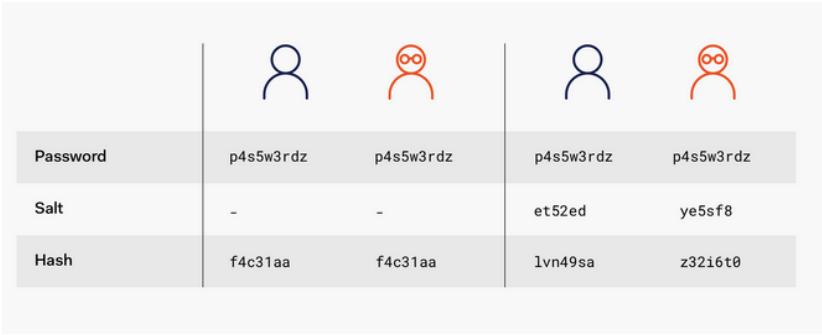
$ echo "CiberSeguridad" | md5sum
d5c4752e4a718ee4ad8fc8ba81a833e2  -

$ md5sum Parrot-security-5.0.1_amd64.iso
74ca72645896f83a65acd35ade46b0e0  Parrot-security-5.0.1_amd64.iso
```

Usos de funciones de hash

Las funciones de HASH se usan en lugares criticos de la criptografía moderna:

- Firma digital
- Almacenamiento seguro de contraseñas



Password	p4s5w3rdz	p4s5w3rdz	p4s5w3rdz	p4s5w3rdz
Salt	-	-	et52ed	ye5sf8
Hash	f4c31aa	f4c31aa	lvn49sa	z32i6t0

Problemas con hashes:

- Posibilidad de que haya colisiones (peor si las mismas se pueden manipular)
- Velocidad de su calculo: La rapidez hace que sea vulnerable a ataques de fuerza bruta

Colisiones en función de HASH

```
$md5sum ship.jpg
253dd04e87492e4fc3471de5e776bc3d  ship.jpg

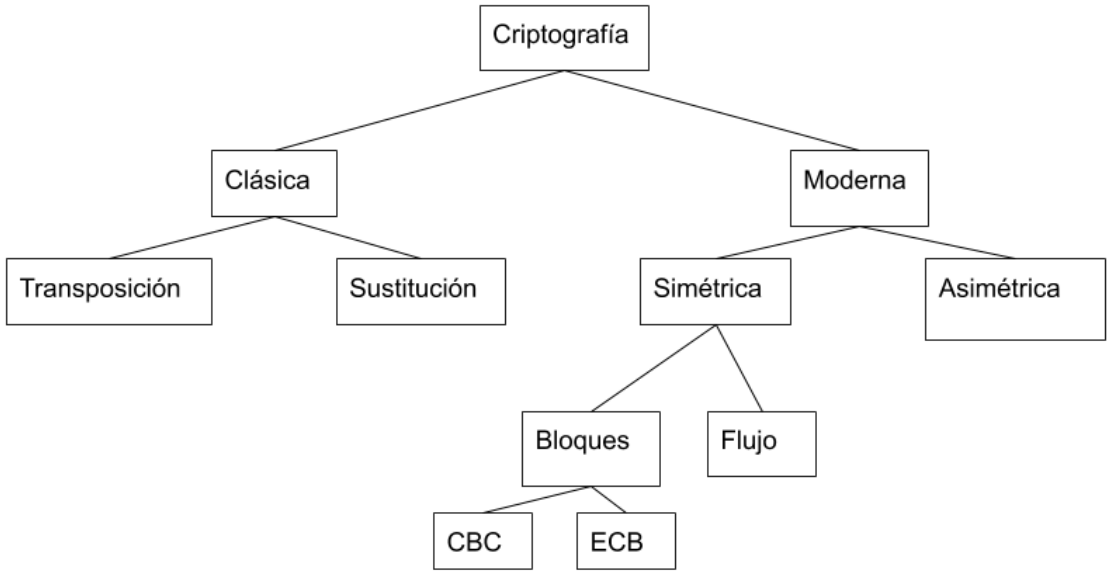
$ md5sum plane.jpg
253dd04e87492e4fc3471de5e776bc3d  plane.jpg
```





> https://natmchugh.blogspot.com/2015/02/create-your-own-md5-collisions.html

Volviendo... Sistemas de criptografía

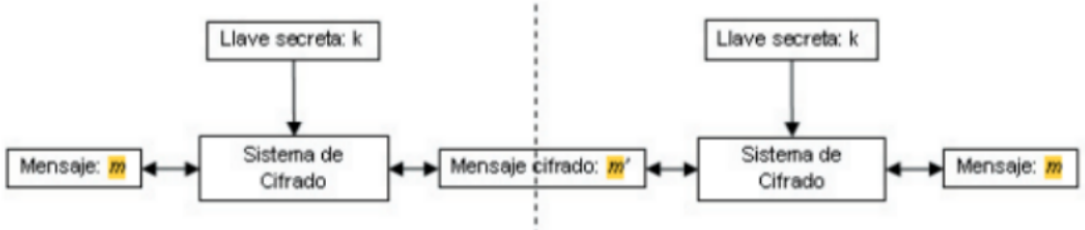



```
Sistemas de cifrado simétrico (también conocidos como sistemas de clave secreta o clave privada)
Sistemas de cifrado asimétrico (también conocidos como sistemas de clave pública)
```

Criptografía simétrica

Emisor:

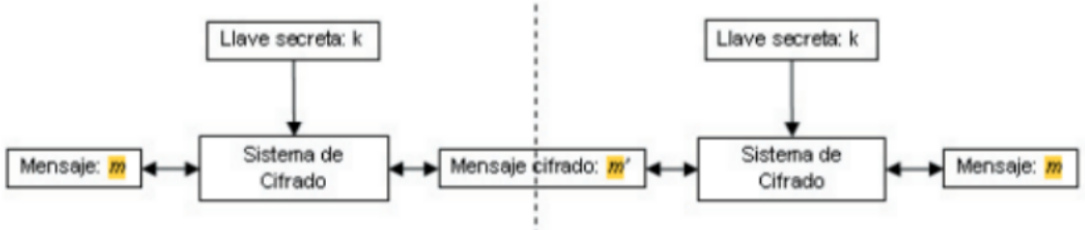
- Genera la clave compartida
- Distribuye la clave compartida
- El mensaje original es cifrado usando la clave compartida
- Se obtiene como resultado un mensaje cifrado
- Envía el mensaje cifrado al destinatario



Criptografía simétrica

Receptor:

- El receptor descifra utilizando el mismo sistema de cifrado y la clave compartida
- Se obtiene como resultado el mensaje original.



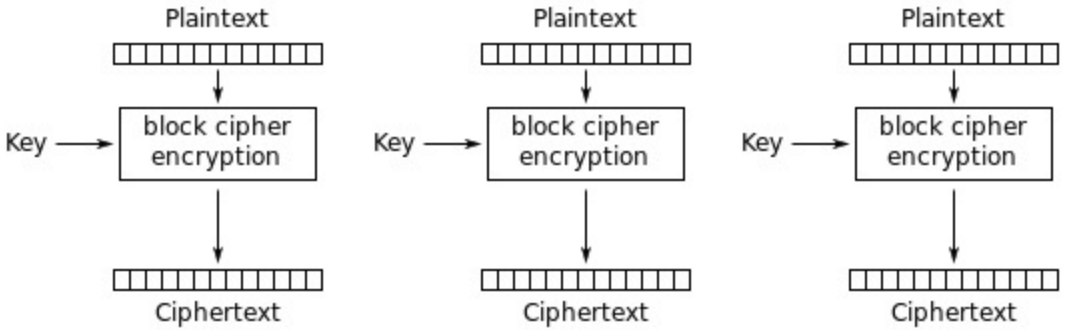
Criptografía simétrica

- Los sistemas simétricos o de clave privada utilizan la misma clave para encriptar y desencriptar
- Existen dos modos de operación básicos:
  - Cifrado en bloques
  - Cifrado de flujo

Cifrado en bloque

- El mensaje en texto claro se divide en bloques de longitud fija (8,16, … bytes) y luego se aplica el algoritmo de cifrado a cada bloque utilizando una clave secreta. Ejemplos: DES, AES.
- Existen distintos modos de operación dependiendo de cómo se mezcla la clave con el texto claro:
  - ECB: Electronic Codebook
  - CBC: Cipher Block Chaining
  - CFB: Cipher FeedBack
  - OFB: Output FeedBack

Simétrica por bloques: AES ECB



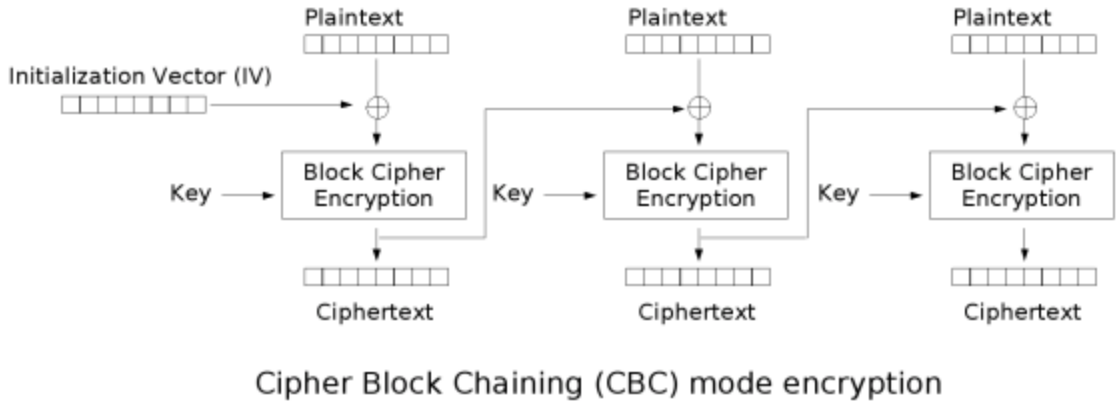
Electronic Codebook (ECB) mode encryption

El mensaje se divide en bloques, y cada uno de ellos es cifrado con la misma clave

¿Desventaja?

- Mismo texto plano mismo texto cifrado

Simétrica por bloques: AES CBC

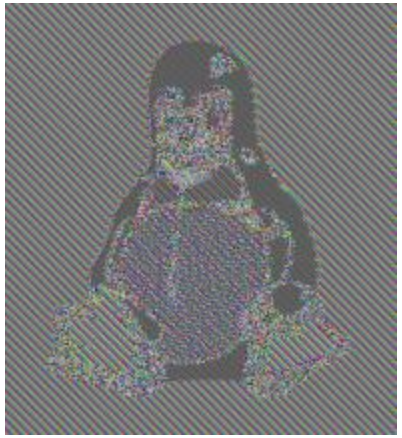
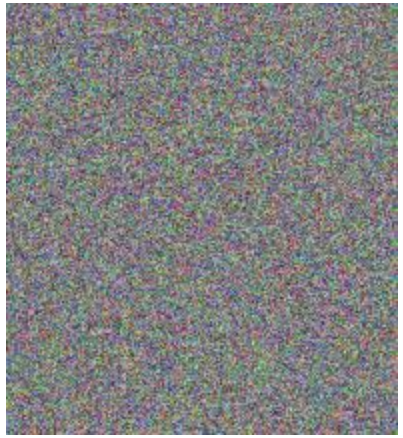


Cipher Block Chaining (CBC) mode encryption

A cada bloque se le aplica XOR con el bloque anterior antes de ser cifrado, cada bloque depende de todo lo procesado anteriormente

Cifrados en bloque

- ECB puede revelar patrones en los datos cifrados

Original	ECB	CBC
		

Cifrado de flujo

- Para algunas aplicaciones, como el cifrado de conversaciones telefónicas, el cifrado en bloques es inapropiada porque los datos se producen en tiempo real en pequeños fragmentos. Las muestras de datos pueden ser tan pequeñas como 8 bits o incluso de 1 bit.
- El algoritmo genera una secuencia pseudoaleatoria (secuencia cifrante o keystream en inglés) de bits que se emplea como clave. El cifrado se realiza combinando la secuencia cifrante con el texto claro. Ejemplo: RC4.



Cifrado simétrico

- Ventajas
  - Gran velocidad de cifrado y descifrado de datos
  - No aumenta el tamaño del mensaje al cifrar datos
- Desventajas
  - La seguridad depende de un secreto compartido entre el emisor y el receptor
  - Problemas en el uso de una misma clave en un grupos de personas
  - La administración de claves no es “escalable”. Se necesita un medio seguro para el intercambio de la clave

Algunos ejemplos de algoritmos: 3DES, RC5, IDEA, AES, Blowfish